

Offline capabilities in UI5

Let's discuss



Viktor Sperling

Senior Developer
SAP SE



Lars Kissel

Senior Developer
SAP SE

Agenda

- 1 Our motivation
- 2 Use Cases for Offline Capabilities
- 3 Aspects of Offline Complexity
- 4 Approaches for Offline Support
- 5 Pain Points
- 6 Your opinion
- 7 Q&A

Our motivation!



Poll 1



menti.com – Code: **8707 4783**

A black smartphone mockup displaying a poll interface. The screen has a white background with a blue header bar. The poll title 'Describe your offline scenario' is in a dark grey font. Below it is a text input area with a light grey border and a small 'x' icon on the right. Inside the input area, there is a hint: 'Short responses are recommended. You have 200 characters left.' At the bottom of the screen is a blue rounded rectangular button with the word 'Submit' in white. The phone's notch and side buttons are visible.

Describe your offline scenario

Short responses are recommended. You have 200 characters left.

Submit

Use Cases



Inspection & checklist completion

- Single User
- Exclusive Data
- Planned offline periods
- Simple-to-dynamic form



Work during travels

- Unpredictable connectivity
- Large or mixed data scope
- Query and filter interactions

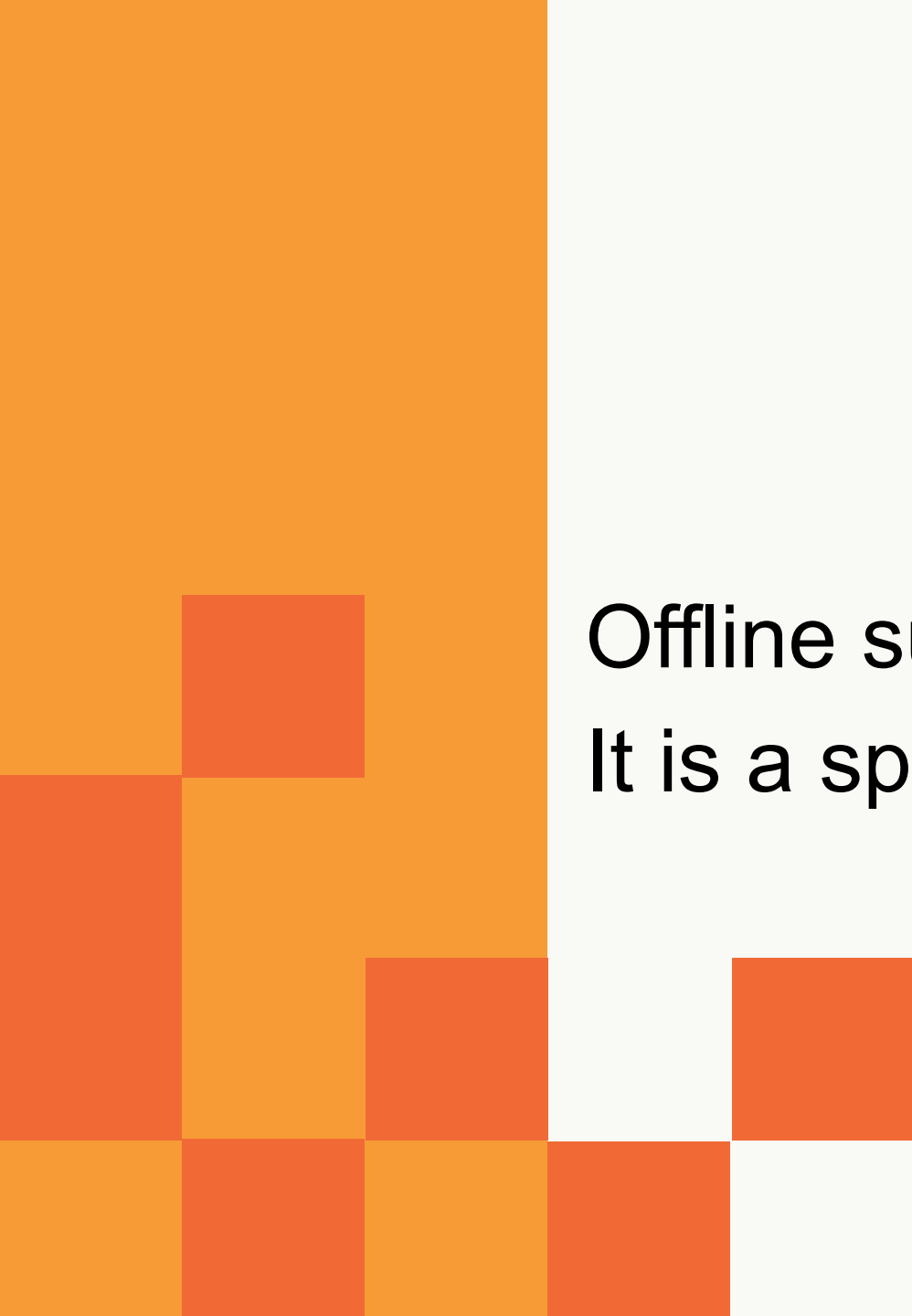


Time critical use cases

- Shared mutable data
- Time-critical writes
- High conflict sensitivity
- Minutes-level staleness

There are use cases which are unfit
for offline!



A decorative graphic on the left side of the slide consists of a grid of orange squares. The top row is a solid orange bar. Below it, a series of squares of varying shades of orange (light, medium, and dark) are arranged in a stepped pattern, creating a staircase effect that descends from the top left towards the bottom right.

Offline support is not a feature.
It is a spectrum of complexity.

Aspects of Offline Complexity



Technical Aspects

Preparation

predictable  spontaneous

Interaction complexity

simple form  business logic

Conflict sensitivity

exclusive  shared

Time Aspects

Offline period

brief  days

Data freshness

minutes  days

Time-criticality

tolerant  critical

Different approaches of how to support offline scenarios

One app

Three implementation ideas



OData V4 Model as Change Buffer

Idea 1

Goal

- Help the user send requests at the right moment: don't submit changes while offline

Offline aspects

- Unprepared, unintentional connection loss
- Short-term offline (seconds to minutes)

Scenario

- Commuter on a train, signal drops between two stations

OData V4 Model as Change Buffer

Idea 1 – How it works

Manage data flow

- Preload required data
- Keep changes locally
 - Use application groups
 - v4.ODataModel parameter: updateGroupId
- Offline? Disable controls that can't succeed
- Online? Allow submitting changes
 - v4.ODataModel#submitBatch

Check connectivity

- Network status
 - Navigator.onLine
- Ping request
 - Window.fetch, AbortController
 - Reliable feedback whether a server is reachable

Service Worker as OData Proxy

Idea 2

Goal

- Add offline support without changing the app: the Service Worker handles it in background

Offline aspects

- Any technical offline aspect
- Mid-term offline (minutes to hours)

Scenario

- Field worker visiting customers, connection comes and goes

Service Worker as OData Proxy

Idea 2 – How it works

Offline preparation

- Preload all required entity sets
- Store data in IndexedDB

Online?

- Forward all requests to the back end
- Keep IndexedDB up to date when changing data

Switch from offline to online

- Send all changes in a combined \$batch to the back end

Offline?

- Parse \$batch requests
 - Read requests:
 - Parse query options
 - Find matching resource(s)
 - Change requests:
 - Apply changes to IndexedDB
 - Maintain a changelog
- Build \$batch response

JSONModel with Explicit Sync

Idea 3

Goal

- Give the user an explicit offline workflow: download, work, submit

Offline aspects

- Prepared, intentional offline session
- Long-term offline (hours to days)

Scenario

- Traveler on a flight for several hours, working in airplane mode

JSONModel with Explicit Sync

Idea 3 – How it works

Build client-side app

- Use JSONModel as default model on UI
- Filtering and sorting supported out of the box

Synchronize data

- Use a v4.ODataModel to download all required entity sets
- Set data to JSONModel

Handle changes

- Maintain changes in a changelog
- Submit changes with v4.ODataModel
 - v4.ODataListBinding#create
 - v4.Context#setProperty
 - v4.Context#delete

Replicated local OData Service

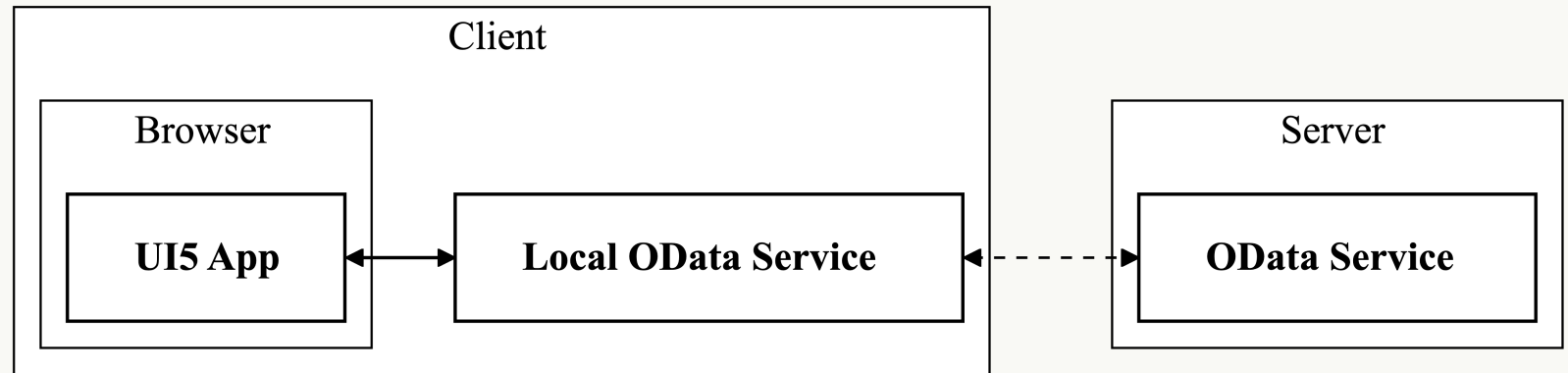
Alternative Idea

Goal

- Give the app a full OData service on the client

What you get

- App code unchanged
- A real OData service
- Knowledge about OData protocol out of the box

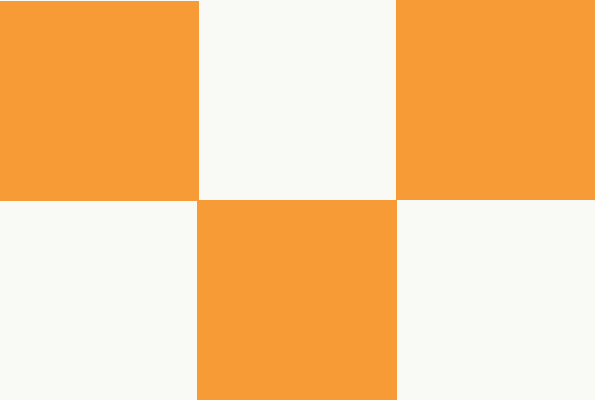


Pain Points

The more complex the app,
the more effort offline support requires.



Pain Points



Back-end logic in the client

- Understand the OData protocol
 - Query options like \$filter quickly become complex
- Navigation Properties
 - Resolve references locally
- Repeat business logic implementation
 - OData Actions
 - Server-side calculated fields

Conflicts at sync time

- Concurrent modifications, or business rule violations
 - Concept for conflict resolution needed
- Expired authentication or session tokens
 - Refresh before sync

Poll 2



menti.com – Code: **4800 8119**

1 OData V4 change buffer

- Buffer requests locally
- Use application group IDs for that
- Determine offline/online mode via pings
- Allow submitting only when online

2 Service Worker as OData proxy

- Preload and store data in IndexedDB
- SW intercepts requests
- Online: forwards the request
- Offline: Parses batch requests and responds to app

3 JSONModel with explicit sync

- Preload data via V4 model
- Sync it with JSONModel
- Offline: Keep change log of changes in the JSONModel
- Online: Submit changes via V4 model
- Supports filtering and sorting

Thank you!

You want to talk to us?



Join our Expert Corner!

#2 Offline Capabilities in UI5



11:50 – 12:50

Scan and share your thoughts!

Offline capabilities in UI5 – Let's discuss

A smartphone is shown vertically, displaying a survey interface. The top of the screen features a header with the text 'UI5con 14. July 2026 St. Leon-Rot, Germany' on an orange background. Below this, the survey text reads: 'We're eager to hear your insights! This survey aims to gather **your feedback on the above named session or workshop at UI5con 2026.** Your responses will help the presenter understand what worked well and what they can improve in future events. Thank you for taking the time to participate!'. The survey question is 'How do you rate the session? *'. Below the question is a vertical list of radio buttons numbered 1 through 8. The word 'poor' is placed to the left of the first radio button (1).

UI5con 14. July 2026
St. Leon-Rot, Germany

We're eager to hear your insights! This survey aims to gather **your feedback on the above named session or workshop at UI5con 2026.**

Your responses will help the presenter understand what worked well and what they can improve in future events.

Thank you for taking the time to participate!

How do you rate the session? *

poor

1 ☐

2 ☐

3 ☐

4 ☐

5 ☐

6 ☐

7 ☐

8 ☐